nordhjem store · guide

Semantic search

The Python code behind the search box,
one function at a time

This guide walks through the search as it is now in the repository: `api/models.py`, `api/search.py` and `api/parse_query.py`. Every code block is the real code, and every number was measured on the shop's 500 products. Section 1 explains the ideas without code; the rest follows the code, one function at a time.

contents

1 What semantic search is **3**
- 1.1 The problem with matching words . . . 3
- 1.2 Searching by meaning . . . 3
- 1.3 Meaning as a list of numbers . . . 3
- 1.4 Closeness as a score . . . 4
- 1.5 How it works: two phases . . . 4
- 1.6 Meaning and rules . . . 5
- 1.7 The words used in this guide . . . 5
- 1.8 From ideas to code . . . 5

2 The data: products as numbers **6**
- 2.1 One product . . . 6
- 2.2 Turning text into numbers . . . 6
- 2.3 Two kinds of text . . . 6

3 The index: all products in memory **7**
- 3.1 The `Index` class . . . 7
- 3.2 `load_index`: reading Firestore once . . . 7

4 The question as numbers **8**
- 4.1 `get_embed_query` . . . 8
- 4.2 `normalise`: making every vector length 1 . . . 9

5 The score: one number per product **9**

6 Filters: rules, not meaning **10**
6.1 `SearchFilters` . . . 10
6.2 `filter_products`: one True or False per product . . . 10

7 Putting it together: `search` **11**
7.1 `np.where`: removing without deleting . . . 11
7.2 Capping `k` . . . 12
7.3 `get_top_k` . . . 12
7.4 The result . . . 12

8 Measuring it **12**

9 Filters from the words **13**
9.1 The problem . . . 13
9.2 `ParsedQuery`: the form the model fills in . . . 13
9.3 `parse_query`: the call . . . 14
9.4 `INSTRUCTION`: what the fields mean . . . 14
9.5 `merge_filters`: when a click and the words disagree . . . 15
9.6 Where it is called . . . 15
9.7 Results . . . 16

10 Adding a filter: colour **16**

11 What it cannot do yet **17**

A Where everything is **17**

1 What semantic search is

1.1 *The problem with matching words*

The simplest search compares *words*. It counts how many words of the question appear in each product's text, and shows the products with the most. This is a *keyword search*. The shop started with one, in `api/keyword_search.py`.

keyword search: a search that matches the letters of words, not what they mean.

Here it is on a real question:

> *a couch my guests can sleep on*

Rank	Product	Shared words
1	KILSHOLM sofa (a sofa bed)	a, guests, …
2	EGEEVIG sofa (a sofa bed)	a, guests, …
3	LUNDSTED rug	a, can, on

Table 1 The keyword search. A rug comes third, because its text "… can be used outdoors on a terrace" shares three small words with the question.

Two things go wrong. The word that matters, "couch", is in no product text at all: the shop says "sofa". And small words such as "a", "can" and "on" count as much as the important ones. A keyword search sees spelling. It does not see meaning.

1.2 *Searching by meaning*

Semantic search compares what the question *means* with what each product *is*. "couch" and "sofa" mean the same thing; "my guests can sleep on" means "folds out into a bed for guests". A semantic search finds the sofa beds, and the rug drops to the bottom.

Semantic search: a search that finds texts with the same meaning, even when they use different words.

It needs a way to put meaning into a form a computer can compare. That form is a list of numbers.

1.3 *Meaning as a list of numbers*

An *embedding model* reads a text and returns a fixed-length list of numbers. Here the model is Google's `gemini-embedding-001`, and the list has 768 numbers. Such a list is called a *vector*, and the list for a text is its *embedding*.

embedding model: a model trained on huge amounts of text to turn any text into a list of numbers, so that similar meanings give similar numbers.

No single number means "sofa" or "soft". What matters is the whole list, seen as a point in space. A map is a good picture: a city's position takes two numbers, latitude and longitude, and cities close on the map have close numbers. An embedding is a position with 768 numbers instead of 2, and texts with close meanings get close positions. "couch for guests", "sofa bed" and "Schlafsofa" land near each other; "wool rug" lands far away.

1.4 *Closeness as a score*

To compare two positions, the search measures the *angle* between the two vectors, as seen from zero. Pointing the same way means the same meaning. This is the *cosine similarity*: a score near 1 is close, lower is further.

cosine similarity: a number for how closely two vectors point the same way: 1 is the same direction, 0 is unrelated.

When every vector is first scaled to length 1 (*normalised*), the cosine is simply the dot product: multiply the two lists pair by pair and add up. That is one line of NumPy, `@`.

1.5 *How it works: two phases*

A semantic search is split into work done once, in advance, and work done on every search.

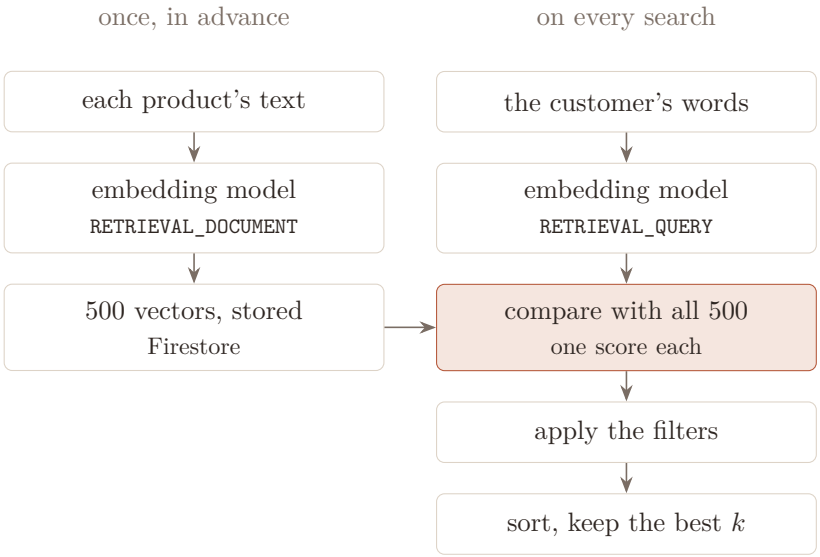


Figure 1 The two phases. The products are embedded once; only the question is embedded on each search.

Once, in advance (*indexing*). Every product's text is sent to the embedding model, and the 500 vectors are stored. This is slow and costs a little money, so it happens once, and again only when a product changes.

On every search (*querying*). Five steps:

1. **Embed the question**: the customer's words become one vector, with the same model, so it lands in the same space as the products.
2. **Normalise** it to length 1, like the stored products.
3. **Score**: compare the question with every product; each gets one cosine score.
4. **Filter**: remove products that break a hard rule (wrong category, too expensive, wrong colour).
5. **Rank**: sort by score and return the best k (the *top-k*).

1.6 *Meaning and rules*

Some wishes are about meaning: “comfy”, “for guests”, “survives a dog”. Others are rules with a yes or no answer: “a sofa”, “green”, “under 3000 kr”. An embedding is good at the first kind and bad at the second: it cannot tell that 8 969 is more than 4 000. So the search uses both:

	Meaning	Rules
Example	comfy, for guests, cosy	sofa, dark green, at most 3000 kr
Handled by	the embedding: it *ranks*	a filter: it *removes*
Answer	a score, more or less	yes or no
In the code	`index.vectors @ question`	`filter_products`

Table 2 Two kinds of wish, two tools.

Rules can come from a click (the Sofa chip, the price menu) or from the words themselves (“under 3000”). Reading them from the words needs a second, different model: a text model that fills in a form (section 9).

1.7 *The words used in this guide*

Word	Meaning here
embedding, vector	the 768 numbers that stand for a text’s meaning
embedding model	the model that makes them (`gemini-embedding-001`)
text model	a model that reads and writes text (`gemini-3-flash-preview`)
normalise	scale a vector to length 1, keeping its direction
cosine similarity, score	how close two meanings are; higher is closer
task type	a hint to the embedding model: question or document
index	all product vectors and fields, in memory, ready to compare
filter, mask	a rule; the True/False list that applies it
top-k	the k best products, best first
structured output	a text model filling in a form instead of writing prose
hit@5, precision@5	how the search is measured (section 8)

1.8 *From ideas to code*

Each step is one Python function:

Step	Function	Section
store the products (done once)	`loaders/load_products.py`	2
read them into memory	`load_index`	3
embed the question	`get_embed_query`	4
normalise	`normalise`	4
score	`index.vectors @ question`	5
filter	`filter_products`, `np.where`	6, 7
rank	`get_top_k`	7
all of it, in order	`search`	7
read rules from the words	`parse_query`, `merge_filters`	9

The rest of the guide follows that order, with the real code and real numbers. Two searches are followed throughout: “a couch my guests can sleep on” and “green sofa that is comfy, less than 3000”.

2 The data: products as numbers

2.1 One product

The shop has 500 products in `data/products.jsonl`, one per line. Here is the first one, shortened:

```
{"product_id": "P1001", "name": "KILGAARD mattress",
 "category": "mattress", "colour": "grey", "price_dkk": 1489.0,
 "description": "Memory foam mattress with medium firmness,
                 180x200 cm. Cooling cover that keeps you from
                 getting too warm at night."}
```

2.2 Turning text into numbers

A computer cannot compare meanings directly. So each product’s text is turned into an *embedding*: a list of 768 numbers. The model that does this is `gemini-embedding-001`, run on Google Cloud.

embedding: a list of numbers that stands for the meaning of a text. Texts with similar meaning get similar lists.

This happened *once*, before any search, in `loaders/load_products.py`. For each product it builds one sentence from the name, category, colour and description, and embeds it:

```
return (f"{product['name']}. Category: {category}. "
        f"Colour: {product['colour']}. {product['description']}")
```

The 500 lists of numbers are stored in Firestore, the shop’s database, in the collection `products`, one document per product, in a field called `embedding`.

2.3 Two kinds of text

The loader embeds products with the task type `RETRIEVAL_DOCUMENT`; the search embeds the customer’s words with `RETRIEVAL_QUERY`. A *task type* tells

task type: a hint to the embedding model about what the text is for, so it can place it better.

the model which side of the search the text is on: a product *describes a thing*, a question *describes a need*. Using the query type for questions was measured on the 20 test questions:

Questions embedded as	precision@5	gap to the first wrong product
`RETRIEVAL_QUERY` (used)	0.98	+0.043
`RETRIEVAL_DOCUMENT`	0.90	+0.021

Table 3 The right task type finds more right products, and separates them better from the wrong ones.

3 The index: all products in memory

3.1 The `Index` *class*

The search needs five things about every product: its ID, its numbers, its category, its colour and its price. They are kept together in one *dataclass*:

dataclass: a Python class that only holds values; Python writes the `__init__` for you.

```
@dataclass
class Index:
    """The products as NumPy arrays; row i of every array is product i."""

    ids: list[str]
    vectors: np.ndarray      # shape (n, 768), every row length 1
    categories: np.ndarray   # shape (n,), text
    colours: np.ndarray      # shape (n,), text
    prices: np.ndarray       # shape (n,), DKK
```

The rule that holds it together is in the docstring: *row i of every array is product i*. If `ids[7]` is `"P1008"`, then `vectors[7]` are its numbers, `categories[7]` its category, and so on. Every step later relies on that.

`Index` is the *class*, the blueprint. `index = Index(...)` makes one *instance*, filled with real values. You read from the instance: `index.prices`, never `Index.prices`.

3.2 `load_index`*: reading Firestore once*

```
@lru_cache(maxsize=1)
def load_index() -> Index:
    db = firestore.Client(project=PROJECT)
    idd, vector, category, price, colour = [], [], [], [], []
    for document in db.collection("products").stream():
        product = document.to_dict()
        idd.append(document.id)
        vector.append(list(product["embedding"]))
        category.append(product["category"])
        price.append(product["price_dkk"])
        colour.append(product["colour"])
```

```
    store_index = Index(ids=idd, vectors=np.array(vector),
                        categories=np.array(category),
                        prices=np.array(price),
                        colours=np.array(colour))
    return store_index
```

Line by line:

- `db.collection("products").stream()` hands over the 500 documents one at a time.
- `document.to_dict()` turns one document into a Python dictionary, so `product["category"]` works.
- Each field goes into its own list. Because all five lists grow together, row i stays product i.
- `np.array(vector)` turns the list of 500 lists into one table of 500 rows and 768 columns. That is 3 MB in memory.

`@lru_cache` makes this run only on the first search. Reading Firestore takes a few seconds; after that, every search reuses the same `Index`.

`@lru_cache`: remembers what a function returned. The next call with the same arguments returns the saved result without running the function.

> The field names in Firestore come from the data, not from your code. The data says `"colour"` (British spelling), so `product["color"]` raises a `KeyError`, whatever you call your own variables.

4 The question as numbers

4.1 `get_embed_query`

```
def get_embed_query(text: str) -> np.ndarray:
    result = get_client().models.embed_content(
        model=EMBED_MODEL,
        contents=[text],
        config=types.EmbedContentConfig(
            task_type="RETRIEVAL_QUERY",
            output_dimensionality=DIMENSIONS,
        ),
    )
    return np.array(result.embeddings[0].values)
```

`contents` is a list because the API can embed many texts in one call; here there is one. So the answer is `result.embeddings[0]`, and `.values` are its 768 numbers. `output_dimensionality=768` asks for the short version: the model's full size is 3072 numbers.

For "a couch my guests can sleep on" the first four of the 768 are:

```
[-0.0014, -0.0080, -0.0385, -0.0627, ...]      length 0.5905
```

4.2 `normalise`: *making every vector length 1*

The *length* of that vector is 0.59, not 1. The model only returns length-1 vectors at its full size; shortened to 768 numbers they come out shorter. The products in Firestore were already divided by their length in the loader, so the question must be too:

length: of a vector: $\sqrt{x_1^2 + x_2^2 + \ldots}$, the straight-line distance from zero, as in Pythagoras.

```
def normalise(vectors: np.ndarray) -> np.ndarray:
    vector_norm = np.linalg.norm(vectors, axis=-1, keepdims=True)
    normalised_vector = vectors / vector_norm
    return normalised_vector
```

```
before   [-0.0014, -0.0080, -0.0385, -0.0627, ...]   length 0.5905
after    [-0.0024, -0.0135, -0.0653, -0.1062, ...]   length 1.0
```

Every number is divided by 0.5905, so the direction stays the same and only the length changes.

The two arguments make the same function work on one vector and on a whole table:

- `axis=-1`: measure along the *last* axis, so a table gets one length per row, not one length for everything.
- `keepdims=True`: keep the result as a column (500 rows, 1 column), so that `vectors / vector_norm` divides each row by its own length.

5 The score: one number per product

```
similarity_scores = index.vectors @ normalised_embedding_query
```

`@` is matrix multiplication. A table of 500 rows times one vector gives 500 numbers: for each product, its 768 numbers multiplied with the question's 768 numbers, pair by pair, and added up. That sum is the *dot product*.

dot product: multiply two lists pair by pair and add up the results.

Because both sides have length 1, the dot product equals the *cosine similarity*. That is the score you see on the page:

cosine similarity: how closely two vectors point the same way: 1 means the same direction, 0 means unrelated.

Score	Product	Rank
0.716	HALDAGAARD sofa (a sofa bed)	1
0.715	HALDAGAARD sofa, other colour	2
0.712	HALDAGAARD sofa, other colour	3
0.572	LUNDSHOLM pillow	251
0.530	SKOVELØSE kontorstol (an office chair)	500

Table 4 "a couch my guests can sleep on": scores from the top, the middle and the bottom of the 500.

Two things to notice. Even the worst product scores 0.53, not 0: all furniture

texts are somewhat alike. And the gap between a right and a wrong answer is small (0.72 against 0.57). A score is only useful *compared with the other scores of the same search*, never as a fixed pass mark.

6 Filters: rules, not meaning

6.1 `SearchFilters`

A score ranks; it never removes anything. A filter does the opposite: it removes every product that breaks a rule. The filters travel in one object, defined in `api/models.py`:

```
Category = Literal[
    "sofa", "mattress", "rug", "dining_table",
    "office_chair", "garden", "pillow", "duvet",
]
Colour = Literal[
    "blue", "rust", "dark green", "beige",
    "grey", "black", "white",
]

class SearchFilters(BaseModel):
    category: Category | None = None
    colour: Colour | None = None
    max_price: float | None = Field(default=None, gt=0)
```

- `Literal[...]`: the value must be one of these exact words. `"bed"` or `"green"` would be rejected.
- `| None = None`: when the customer does not use a filter, it is `None`, and it removes nothing.
- `gt=0`: a price must be above zero.

`SearchFilters(category="sofa", max_price=4000)` means "only sofas, at most 4000 kr". `SearchFilters()` means "no filters".

6.2 `filter_products`*: one True or False per product*

```
def filter_products(index: Index, filters: SearchFilters) -> np.ndarray:
    index_length = len(index.categories)
    filtered_array = np.ones(index_length, dtype=bool)
    if filters.category is not None:
        filtered_array = filtered_array & (index.categories == filters.category)
    if filters.max_price is not None:
        filtered_array = filtered_array & (index.prices <= filters.max_price)
    if filters.colour is not None:
        filtered_array = filtered_array & (index.colours == filters.colour)
    return filtered_array
```

It starts with `np.ones(..., dtype=bool)`: 500 times `True`, every product allowed. Each filter that is set then turns some of them to `False`.

`index.categories == "sofa"` compares all 500 at once and gives 500 True or False values. `&` combines two such arrays position by position: the result is `True` only where *both* are `True`. On four products:

```
categories    ["sofa", "sofa", "rug", "sofa"]
prices        [3000,   6500,   1200,  2500]

start         [True,   True,   True,  True]
& sofa?       [True,   True,   False, True]
& <= 4000?    [True,   False,  False, True]    <- returned
```

On the real shop, for "green sofa under 3000":

```
start                 500 products allowed
& category == sofa     90
& colour == dark green 14
& price <= 3000         1      (VESTEBY sofa, 2 639 kr)
```

> The brackets in `filtered_array & (index.prices <= 4000)` are needed. In Python, `&` binds more tightly than `<=`, so without them Python would compute `filtered_array & index.prices` first.

7 Putting it together: `search`

```python
def search(query: str, filters: SearchFilters, k: int) -> list[Hit]:
    index = load_index()
    embedding_query = get_embed_query(query)
    normalised_embedding_query = normalise(embedding_query)
    similarity_scores = index.vectors @ normalised_embedding_query
    mask = filter_products(index, filters)
    scores = np.where(mask, similarity_scores, -np.inf)
    k = min(k, int(mask.sum()))  # never more hits than allowed products
    top_k = get_top_k(scores, k)
    return [Hit(product_id=index.ids[row], score=float(scores[row]))
            for row in top_k]
```

The first four lines are sections 3 to 5. The rest is new.

7.1 `np.where`: *removing without deleting*

`np.where(mask, similarity_scores, -np.inf)` goes through the 500 positions. Where `mask` is `True`, it keeps the score; where it is `False`, it puts $-\infty$ (minus infinity) instead:

```
mask      [True,  False, False, True ]
scores    [0.71,  0.69,  0.55,  0.68 ]
```

```
result    [0.71,  -inf,  -inf,  0.68 ]
```

Why not delete the filtered products? Because then row i would no longer be product i, and `index.ids[row]` would point at the wrong product. With $-\infty$, every array keeps its 500 rows, and a filtered product can never beat an allowed one.

7.2 *Capping* `k`

The customer asks for 24 results, but only one green sofa costs 3000 kr or less. Without the cap, the other 23 places would be filled with $-\infty$ products: products that broke the filter. `mask.sum()` counts the `True` values (`True` counts as 1), so `min(k, int(mask.sum()))` never asks for more than are allowed.

7.3 `get_top_k`

```python
def get_top_k(scores: np.ndarray, k: int) -> list[int]:
    return np.argsort(scores)[::-1][:k]
```

Read it from left to right:

```
scores                 [0.2, 0.9, 0.1, 0.7, 0.5]
np.argsort(scores)     [2, 0, 4, 3, 1]    rows, lowest score first
[::-1]                 [1, 3, 4, 0, 2]    reversed: highest first
[:3]                   [1, 3, 4]          the first k
```

`argsort` returns *row numbers*, not scores. That is what the last line of `search` needs: `index.ids[row]` for the ID, `scores[row]` for the score.

7.4 *The result*

`search` returns a list of `Hit`, each a product ID and its score. `float(...)` turns NumPy's number type into a plain Python number, so it can be sent as JSON:

```
product_id='P1199' score=0.7156
product_id='P1198' score=0.7151
product_id='P1200' score=0.7125
```

8 Measuring it

"It looks right" is not a test. `steps/01_embeddings/check.py` runs 23 questions from `data/eval/product_queries.json`. Each question lists the products that are right answers. Two numbers are computed on the first five results:

- *hit@5*: did the customer see anything useful?
- *precision@5*: how much of the first screen is useful?

hit@5: 1 if at least one of the first five results is right, else 0.
precision@5: the share of the first five results that are right.

Kind of question	hit@5	precision@5	Example
meaning	1.00	0.99	a couch my guests can sleep on
language	1.00	0.96	Schlafsofa für Gäste
all graded	1.00	0.98	

Table 5 The Step 1 search on its test questions. The pass marks were 0.95 and 0.90.

The *language* questions are in Danish or German while most product texts are in English. The embedding model places them in the same spot anyway: meaning, not words.

Three more questions are kept apart, marked `needs_filter`: "sofa under 4000 kr", "sofa that is not leather" and "mattress 180x200". Meaning alone cannot answer them. For "sofa under 4000 kr", precision@5 was 0.00: five sofas, all too expensive. That led to the next step.

9 Filters from the words

9.1 *The problem*

Type "A very comfortable bed under 4000" with the price menu on *Any*, and the second result costs 8 969 kr. The filter code is not broken: `filters.max_price` is `None`, because nobody clicked a price. The words "under 4000" only went into the embedding, and an embedding cannot compare numbers.

So the sentence is read *before* the search and split in two:

```
"A very comfortable bed under 4000"
   search_text = "A very comfortable bed"   -> the embedding
   category    = "mattress"                -> filter_products
   max_price   = 4000                      -> filter_products
```

9.2 `ParsedQuery`*: the form the model fills in*

This is a job for a *text* model (`gemini-3-flash-preview`), not the embedding model. Left alone, a text model writes sentences. With *structured output* it must fill in a form instead. The form is a Pydantic class, in `api/parse_query.py`:

structured output: the model must reply with data in a shape you define, not with free text.

```
class ParsedQuery(BaseModel):
    search_text: str
    category: Category | None
    colour: Colour | None
    max_price: float | None
```

The same `Category` and `Colour` lists as in `SearchFilters` are reused, so the model can only answer a value the search knows. None of the fields has a default value, so the model must fill in every one, with `null` when there is nothing.

9.3 `parse_query`: *the call*

```
@lru_cache(maxsize=256)
def parse_query(text: str) -> ParsedQuery:
    response = get_text_client().models.generate_content(
        model=TEXT_MODEL,
        contents=text,
        config=types.GenerateContentConfig(
            system_instruction=INSTRUCTION,
            response_mime_type="application/json",
            response_schema=ParsedQuery,
        ),
    )
    return response.parsed
```

- `system_instruction`: what to do (next section).
- `response_mime_type="application/json"`: reply with JSON.
- `response_schema=ParsedQuery`: in exactly this shape.
- `response.parsed`: a ready `ParsedQuery` object, not text to take apart.
- `@lru_cache(maxsize=256)`: the same sentence is only sent to Gemini once, for example when the customer clicks “Show more”.

9.4 `INSTRUCTION`: *what the fields mean*

The schema fixes the *shape* of the answer. Only the instruction fixes its *meaning*. Each rule in it answers one trap from the test sentences in `data/eval/filter_queries.json`:

Rule in the instruction	Test sentence it handles
search_text without the price	“A very comfortable bed under 4000” → “A very comfortable bed”
other words and languages count	“billig *madras*”, “a *couch*”, “*Bürostuhl*”
no category named: null, do not guess	“something cosy for the living room floor”
a size or a count is never a price	“mattress 180x200”, “dining table for 8 people”
with a range, use the upper end	“sofa between 2000 and 5000 kr” → 5000

Table 6 The instruction, rule by rule.

The highlighted row matters most. A filter is a hard cut: a wrong category hides every other product. An empty field costs nothing, because the embedding still finds rugs by meaning.

> The example in the instruction for "do not guess" is *something soft for the bedroom*, not the test sentence itself. If the prompt contains the test sentence, passing the test proves little.

9.5 `merge_filters`: *when a click and the words disagree*

A customer can type "bed under 4000" *and* pick 2500 in the price menu. Then a click wins, field by field, because it is the more deliberate choice:

```
def merge_filters(clicked: SearchFilters,
                  parsed: ParsedQuery) -> SearchFilters:
    category = (clicked.category if clicked.category is not None
                else parsed.category)
    colour = clicked.colour if clicked.colour is not None else parsed.colour
    max_price = (clicked.max_price if clicked.max_price is not None
                 else parsed.max_price)
    return SearchFilters(category=category, colour=colour,
                         max_price=max_price)
```

`a if condition else b` is Python's one-line `if`. The last line calls the *class* `SearchFilters` to make a new instance; calling `clicked(...)` would fail, because `clicked` is an instance, not a class.

9.6 *Where it is called*

`api/app.py` runs the two functions before your `search`:

```
def read_filters(request: SearchRequest) -> tuple[str, SearchFilters]:
    clicked = SearchFilters(category=request.category,
                            max_price=request.max_price)
    try:
        parsed = parse_query.parse_query(request.query)
        filters = parse_query.merge_filters(clicked, parsed)
    except Exception:
        logging.exception("parse_query failed; searching without it")
        return request.query, clicked
    return parsed.search_text or request.query, filters
```

If Gemini fails, the search still runs, with the clicked filters and the original words. Reading the filters is an improvement, never a reason for a search to break.

9.7 Results

Test	Result
category right, 15 test sentences	15 of 15
max_price right, 15 test sentences	15 of 15
“sofa under 4000 kr”, precision@5	0.00 → 1.00
`needs_filter` questions, precision@5	0.60 → 0.93
meaning and language questions	unchanged (1.00 / 0.98)

Table 7 `steps/01b_filters_from_words/check.py`.

10 Adding a filter: colour

“green sofa that is comfy, less than 3000” showed the next gap. Colour only *ranked*: green sofas came first, then white and beige ones filled the page. Making colour a filter touched every layer, which makes it a good map of the code:

File	Change
`api/models.py`	`Colour = Literal[...]` with the 7 colours; `colour` in `SearchFilters`
`api/search.py`	`colours` in `Index`; filled in `load_index`; one more `if` in `filter_products`
`api/parse_query.py`	`colour` in `ParsedQuery`; one line in `merge_filters`; a colour rule in `INSTRUCTION`

Table 8 One new filter, three files.

Two traps came up on the way:

- **“dark green” is one value.** Written as `"dark", "green"`, the list has two colours that match no product. And customers say “green”, so the instruction maps it: *green or olive is dark green, navy is blue, cream is beige.*
- **One spelling everywhere.** With `colour` in one file and `color` in another, Pydantic silently *drops* an unknown field name: `SearchFilters(color=...)` gives no error, and no filter.

After the change, the same search:

```
"green sofa that is confy that cost less than 3000"
 read as:  category='sofa'  colour='dark green'  max_price=3000.0
 VESTEBY sofa   dark green   2 639 kr        (the only one)
```

One result, because the shop really has only one green sofa under 3000 kr. Both checks were extended: the Step 1 check has a “dark green sofa” filter test, and the Step 1b check reads five colour sentences (“grøn sofa”, “navy rug”, “cream coloured duvet”, and one with no colour), all right.

11 What it cannot do yet

- **Material and size** are not filters. “a blue velvet sofa” filters on sofa and blue, but velvet only ranks. The data already has them (`attributes.material`, `attributes.size`); adding them follows the colour recipe.
- **Exclusions** such as “not leather” need a different kind of field: a value to *remove*, not to keep.
- **Exact names** such as “HALDAGAARD” have no meaning to embed. A keyword search next to the semantic one (*hybrid search*) handles them.
- **Speed.** Each new search makes one Gemini call to read the filters, which adds one to two seconds.

The next step is different in kind. Semantic search *returns products*; the customer reads them and decides. RAG (retrieval-augmented generation) reuses this same search over policy texts, and then has a text model *write an answer* from what it found.

A Where everything is

File	What is in it
`api/models.py`	`Category`, `Colour`, `SearchFilters`, `Hit`
`api/search.py`	`Index`, `load_index`, `get_embed_query`, `normalise`, `filter_products`, `get_top_k`, `search`
`api/parse_query.py`	`ParsedQuery`, `merge_filters`, `INSTRUCTION`, `parse_query`
`api/app.py`	`read_filters`, `run_search`: calls the two files above
`loaders/load_products.py`	embeds the 500 products into Firestore (run once)
`data/eval/`	the test questions and sentences
`steps/01_embeddings/check.py`	tests `search.py`
`steps/01b_filters_from_words/check.py`	tests `parse_query.py`

Run both checks from the repository root:

```
python3 steps/01_embeddings/check.py
python3 steps/01b_filters_from_words/check.py
```